

Programmation Multiplateforme

Ionic v6

Table des matières

Créez votre première application Ionic.....	2
Installation.....	2
Créer un nouveau projet.....	2
Tester le nouveau projet.....	3
Structure d'un projet Ionic.....	3
Modifier l'application.....	3
Structure d'un <template>.....	5
Tester l'application sur un émulateur Android.....	6
Les composants.....	6
Exemple presque complet.....	14
Cycle de vie.....	16
Requête HTTP.....	17
Géolocalisation.....	20
Images.....	22
Menu de navigation coulissant.....	23
Affichage de liste.....	28
Stockage local.....	31
Module.....	33
Camera.....	36
Menu de navigation à onglet.....	38
Publier sur Google Play.....	39
Références.....	39

Document écrit par Stéphane Gill

Février 2022



Ce document est soumis à la licence Creative Commons Attribution 3.0 Canada. Permission vous est donnée de copier, modifier et redistribuer des copies de ce document, dans les conditions fixées par la licence, tant que cette note apparaît clairement.

Ionic est un cadre d'applications open source créé par en 2013 par Max Lynch, Ben Sperry, et Adam Bradley. Il est utilisé pour développer des applications pour, entre autres, Android, iOS, Chrome et Windows Phone. *Ionic* permet aux développeurs d'utiliser les fonctionnalités natives de ces plateformes.

Créez votre première application Ionic

Installation

- Installer *Node.js*
- Installer l'interface en ligne de commande *Ionic*

```
npm install -g @ionic/cli
```

Créer un nouveau projet

- Créer un répertoire de travail
- Créer un répertoire contenant l'application *Ionic*

```
mkdir Workspace
```

```
cd Workspace
```

```
ionic start
```

```
1
2 ? Use the app creation wizard? No
3
4 Pick a framework! 😊
5
6 Please select the JavaScript framework to use for your new app. To bypass
7 prompt next time, supply a value for the --type option.
8
9 ? Framework: Vue
10
11 Every great app needs a name! 😊
12
13 Please enter the full name of your app. You can change this at any time.
14 bypass this prompt next time, supply name, the first argument to ionic s
15
16 ? Project name: monProjet
17
18 Let's pick the perfect starter template!
19
20 Starter templates are ready-to-go Ionic apps that come packed with eve
21 you need to build your app. To bypass this prompt next time, supply tem
```

```

22 the second argument to ionic start.
23
24 ? Starter template: blank
25 ✓ Preparing directory ./test2 in 1.10ms
26 ✓ Downloading and extracting blank starter in 972.52ms
27 > ionic integrations enable capacitor --quiet -- test1 io.ionic.starter
28 > npm i --save -E @capacitor/core@latest
29 ...
30 ...
31

```

Tester le nouveau projet

- Démarrer *Ionic*

```
cd MonProjet
```

```
ionic serve
```
- Un navigateur Web s'ouvre et la nouvelle application démarre

Structure d'un projet Ionic

Fichier/répertoire	Description
src	Répertoire contenant le code source de l'application (partie modifiable et partie non modifiable).
views	Répertoire contenant les pages de l'application.
theme/variables.css	Fichier qui permet de modifier les couleurs des thèmes Ionic.
router/index.ts	Fichier qui permet de définir les routes.

Modifier l'application

- Créer un fichier `views/PageUn.vue` contenant le code suivant :

```

1 <template>
2   <ion-page>
3     <ion-content>
4       <div class="ion-text-center milieu">
5         <p>Page 1</p>
6       </div>
7     </ion-content>
8   </ion-page>
9 </template>
10

```

```

11 <script lang="ts">
12 import { IonContent, IonPage } from '@ionic/vue';
13 import { defineComponent } from 'vue';
14
15 export default defineComponent({
16   components: {
17     IonContent,
18     IonPage,
19   }
20 });
21 </script>
22
23 <style scoped>
24 .milieu{
25   height: 100%;
26   display: flex;
27   justify-content: center;
28   align-items: center;
29 }
30
31 </style>

```

Une page vue est composée de 3 sections : <template>, <script> et <style>.

- Modifier le fichier router/index.ts :

```

1 import { createRouter, createWebHistory } from '@ionic/vue-router';
2 import { RouteRecordRaw } from 'vue-router';
3 import PageUn from '../views/PageUn.vue'
4
5 const routes: Array<RouteRecordRaw> = [
6   {
7     path: '/',
8     redirect: '/home'
9   },
10  {
11    path: '/home',
12    name: 'PageUn',
13    component: PageUn
14  }
15 ]
16
17 const router = createRouter({
18   history: createWebHistory(process.env.BASE_URL),
19   routes
20 })
21
22 export default router

```

Structure d'un <template>

- Modifier le fichier `views/PageUn.vue` afin de lui ajouter un entête et un pied de page :

```
1 <template>
2   <ion-page>
3     <ion-header>
4       <ion-toolbar color="primary">
5         <ion-title>Page 1</ion-title>
6       </ion-toolbar>
7     </ion-header>
8
9     <ion-content>
10      <div class="ion-text-center milieu">
11        <p>Page 1</p>
12      </div>
13    </ion-content>
14
15    <ion-footer>
16      <ion-toolbar color="secondary">
17        <ion-title>Pied de page</ion-title>
18      </ion-toolbar>
19    </ion-footer>
20  </ion-page>
21 </template>
22
23 <script lang="ts">
24 import { IonContent, IonHeader, IonPage, IonTitle, IonToolbar,
25                                     IonFooter } from '@ionic/vue';
26 import { defineComponent } from 'vue';
27
28 export default defineComponent({
29   components: {
30     IonContent,
31     IonHeader,
32     IonPage,
33     IonTitle,
34     IonToolbar,
35     IonFooter
36   }
37 });
38 </script>
39
40 <style scoped>
41 .milieu{
42   height: 100%;
43   display: flex;
44   justify-content: center;
45   align-items: center;
46 }
47
48 </style>
```

Tester l'application sur un émulateur Android

- Construire le code exécutable Android

```
cd MonProjet
```

```
npm run build
```

- Démarrer l'application sur l'émulateur

```
ionic capacitor run android -l -external
```

ou

```
npx cap run android
```

C'est vraiment long...

Les composants

Un composant est un élément graphique affiché par l'interface utilisateur. Toutes les pages d'une application sont composées entièrement de composants. Les éléments graphiques, boutons, textes, images, listes, etc., sont des composants.

Exemple 1 : Page d'accueil composé d'un bouton qui redirige vers une autre page.

Créer un nouveau projet puis modifier le fichier `HomePage .vue` dans le répertoire `views`.

```
1 <template>
2   <ion-page>
3     <ion-header>
4       <ion-toolbar color="primary">
5         <ion-title>Home</ion-title>
6       </ion-toolbar>
7     </ion-header>
8
9     <ion-content>
10      <div class="ion-text-center">
11        <p>Page d'accueil</p>
12        <ion-button expand="block" color="medium" href="/pageUn">
13          Redirection</ion-button>
14      </div>
15    </ion-content>
16
17    <ion-footer>
18      <ion-toolbar color="secondary">
19        <ion-title>Pied de page</ion-title>
20      </ion-toolbar>
21    </ion-footer>
```

```

22     </ion-page>
23 </template>
24
25 <script lang="ts">
26 import { IonContent, IonHeader, IonPage, IonTitle, IonToolbar,
27           IonFooter, IonButton } from '@ionic/vue';
28 import { defineComponent } from 'vue';
29
30 export default defineComponent({
31   components: {
32     IonContent,
33     IonHeader,
34     IonPage,
35     IonTitle,
36     IonToolbar,
37     IonFooter,
38     IonButton
39   }
40 });
41 </script>
42
43 <style scoped>
44
45 </style>

```

L'attribut href de la balise <ion-button> (ligne 12) permet la redirection vers une autre page. Créer un fichier PageUn.vue dans le répertoire views.

```

1  <template>
2    <ion-page>
3      <ion-header>
4        <ion-toolbar color="primary">
5          <ion-buttons slot="start">
6            <ion-back-button default-href="/home"></ion-back-button>
7          </ion-buttons>
8          <ion-title>Page 1</ion-title>
9        </ion-toolbar>
10     </ion-header>
11
12     <ion-content>
13       <div class="ion-text-center milieu">
14         <p>Page 1</p>
15       </div>
16     </ion-content>
17
18     <ion-footer>
19       <ion-toolbar color="secondary">
20         <ion-title>Pied de page</ion-title>
21       </ion-toolbar>
22     </ion-footer>
23   </ion-page>
24 </template>
25
26 <script lang="ts">

```

```

27 import { IonContent, IonHeader, IonPage, IonTitle, IonToolbar, IonFooter,
28           IonBackButton, IonButtons } from '@ionic/vue';
29 import { defineComponent } from 'vue';
30
31 export default defineComponent({
32   components: {
33     IonContent,
34     IonHeader,
35     IonPage,
36     IonTitle,
37     IonToolbar,
38     IonFooter,
39     IonBackButton,
40     IonButtons,
41   }
42 });
43 </script>
44
45 <style scoped>
46 .milieu{
47   height: 100%;
48   display: flex;
49   justify-content: center;
50   align-items: center;
51 }
52
53 </style>

```

L'attribut default-href de la balise <ion-back-button> (ligne 6) permet de retourner à la page précédente. À la ligne 13, la classe milieu qui est définie aux lignes 46 à 51 permet de personnaliser la mise en page. Pour permettre la redirection vers cette page, il est nécessaire d'ajouter une route dans le fichier router/index.ts.

```

1  import { createRouter, createWebHistory } from '@ionic/vue-router';
2  import { RouteRecordRaw } from 'vue-router';
3  import HomePage from '../views/HomePage.vue';
4  import PageUn from '../views/PageUn.vue';
5
6  const routes: Array<RouteRecordRaw> = [
7    {
8      path: '/',
9      redirect: '/home'
10   },
11   {
12     path: '/home',
13     name: 'HomePage',
14     component: HomePage
15   },
16   {
17     path: '/pageUn',
18     name: 'PageUn',
19     component: PageUn
20   }

```

```

21 ]
22
23 const router = createRouter({
24   history: createWebHistory(process.env.BASE_URL),
25   routes
26 })
27
28 export default router

```

Exemple 2 : Bouton permettant d'afficher une boîte de dialogue

1. Créer un fichier PageDeux .vue dans le répertoire views.

```

1  <template>
2    <ion-page>
3      <ion-header>
4        <ion-toolbar color="primary">
5          <ion-buttons slot="start">
6            <ion-back-button default-href="/home"></ion-back-button>
7          </ion-buttons>
8          <ion-title>Page 2</ion-title>
9        </ion-toolbar>
10     </ion-header>
11
12     <ion-content>
13       <div class="ion-text-center milieu">
14         <ion-button @click="afficheDialogue">Afficher</ion-button>
15       </div>
16     </ion-content>
17
18     <ion-footer>
19       <ion-toolbar color="secondary">
20         <ion-title>Pied de page</ion-title>
21       </ion-toolbar>
22     </ion-footer>
23   </ion-page>
24 </template>
25
26 <script lang="ts">
27 import { IonContent, IonHeader, IonPage, IonTitle, IonToolbar, IonFooter,
28         IonBackButton, IonButtons, IonButton } from '@ionic/vue';
29 import { alertController } from '@ionic/vue';
30 import { defineComponent } from 'vue';
31
32 export default defineComponent({
33   components: {
34     IonContent,
35     IonHeader,
36     IonPage,
37     IonTitle,
38     IonToolbar,
39     IonFooter,

```

```

40     IonBackButton,
41     IonButtons,
42     IonButton
43 },
44 methods:{
45     async afficheDialogue(){
46         console.log("Mon alerte");
47         const alert = await alertController.create({
48
49             header: 'Mon alerte',
50             subHeader: 'Sous titre',
51             message: 'Ceci est un message d\'alerte',
52             buttons: ['OK'],
53         });
54
55         await alert.present();
56
57         const { role } = await alert.onDidDismiss();
58         console.log('onDidDismiss: ', role);
59     }
60 },
61 });
62 </script>
63
64 <style scoped>
65 .milieu{
66     height: 100%;
67     display: flex;
68     justify-content: center;
69     align-items: center;
70 }
71
72 </style>

```

L'attribut `@click` de la balise `<ion-button>` (ligne 14) permet d'appeler la méthode `afficheDialogue()` lorsque l'utilisateur presse le bouton. La méthode `afficheDialogue()` est définie aux lignes 45 à 59. Elle permet d'afficher une boîte de dialogue modale.

2. Ajouter une route vers le fichier `PageDeux.vue` dans le fichier `router/index.ts`.
3. Ajouter un bouton dans la page `HomePage.vue`. Ce bouton permet de rediriger l'application vers la page `PageDeux.vue`.

Exemple 3 : Formulaire

1. Créer un fichier `PageTrois.vue` dans le répertoire `views`.

```

1 <template>
2   <ion-page>
3     <ion-header>
4       <ion-toolbar color="primary">
5         <ion-buttons slot="start">

```

```

6         <ion-back-button default-href="/home"></ion-back-button>
7     </ion-buttons>
8     <ion-title>Page 3</ion-title>
9 </ion-toolbar>
10 </ion-header>
11
12 <ion-content>
13     <ion-item>
14         <ion-label>Nom: </ion-label>
15         <ion-input v-model="monNom"></ion-input>
16     </ion-item>
17     <ion-item>
18         <ion-label>Genre</ion-label>
19         <ion-select placeholder="Choisir" v-model="monGenre">
20             <ion-select-option value="f">Femme</ion-select-option>
21             <ion-select-option value="h">Homme</ion-select-option>
22         </ion-select>
23     </ion-item>
24     <ion-button @click="afficheDialogue">Afficher</ion-button>
25 </ion-content>
26
27 <ion-footer>
28     <ion-toolbar color="secondary">
29         <ion-title>Pied de page</ion-title>
30     </ion-toolbar>
31 </ion-footer>
32 </ion-page>
33 </template>
34
35 <script lang="ts">
36 import {
37     IonContent,
38     IonHeader,
39     IonPage,
40     IonTitle,
41     IonToolbar,
42     IonFooter,
43     IonBackButton,
44     IonButtons,
45     IonButton,
46     IonLabel,
47     IonItem,
48     IonInput,
49     IonSelect,
50     IonSelectOption
51 } from '@ionic/vue';
52 import { alertController } from '@ionic/vue';
53 import { defineComponent } from 'vue';
54
55 export default defineComponent({
56     components: {
57         IonContent,
58         IonHeader,
59         IonPage,
60         IonTitle,

```

```

61         IonToolbar,
62         IonFooter,
63         IonBackButton,
64         IonButtons,
65         IonButton,
66         IonLabel,
67         IonItem,
68         IonInput,
69         IonSelect,
70         IonSelectOption
71     },
72     methods: {
73         async afficheDialogue() {
74             const alert = await alertController.create({
75
76                 header: 'Mon alerte',
77                 subHeader: 'Votre Nom est:',
78                 message: this.monNom + " " + this.monGenre,
79                 buttons: ['OK'],
80             });
81
82             await alert.present();
83
84             const { role } = await alert.onDidDismiss();
85         },
86     },
87     data() {
88         return {
89             monNom: "",
90             monGenre: ""
91         }
92     }
93 });
94 </script>

```

À la ligne 15, l'attribut `v-model` permet de transférer automatiquement le contenu de la zone de texte (`<ion-input>`) dans l'attribut `monNom` (ligne 89). La boîte de dialogue peut donc récupérer le contenu de cet attribut (ligne 78) et l'afficher.

2. Ajouter une route vers le fichier `PageTrois.vue` dans le fichier `router/index.ts`.
3. Ajouter un bouton dans la page `HomePage.vue`. Ce bouton permet de rediriger l'application vers la page `PageTrois.vue`.

Exemple 4 : Affichage de données

1. Créer un fichier `PageQuatre.vue` dans le répertoire `views`.

```

1  <template>
2    <ion-page>
3      <ion-header>
4        <ion-toolbar color="primary">
5          <ion-buttons slot="start">
6            <ion-back-button default-href="/home"></ion-back-button>
7          </ion-buttons>
8          <ion-title>Page 4</ion-title>
9        </ion-toolbar>
10     </ion-header>
11
12     <ion-content>
13       <ion-item>
14         <ion-label>Nom: </ion-label>
15         <ion-input v-model="monNom"></ion-input>
16       </ion-item>
17       <ion-item>
18         <ion-label>--> </ion-label>
19         <ion-label>{{this.monNom}}</ion-label>
20       </ion-item>
21     </ion-content>
22
23     <ion-footer>
24       <ion-toolbar color="secondary">
25         <ion-title>Pied de page</ion-title>
26       </ion-toolbar>
27     </ion-footer>
28   </ion-page>
29 </template>
30
31 <script lang="ts">
32 import {
33   IonContent,
34   IonHeader,
35   IonPage,
36   IonTitle,
37   IonToolbar,
38   IonFooter,
39   IonBackButton,
40   IonButtons,
41   IonLabel,
42   IonItem,
43   IonInput
44 } from '@ionic/vue';
45 import { alertController } from '@ionic/vue';
46 import { defineComponent } from 'vue';
47
48 export default defineComponent({
49   components: {
50     IonContent,
51     IonHeader,
52     IonPage,
53     IonTitle,
54     IonToolbar,
55     IonFooter,

```

```

56         IonBackButton,
57         IonButtons,
58         IonLabel,
59         IonItem,
60         IonInput,
61     },
62     data() {
63         return {
64             monNom: "",
65         }
66     }
67 });
68 </script>

```

À la ligne 17, l'instruction `{{this.monNom}}` permet d'afficher le contenu de l'attribut `monNom` (ligne 64). À chaque fois que les attributs de la méthode `data()` sont modifiés, l'affichage de la page est généré à nouveau.

2. Ajouter une route vers le fichier `PageQuatre.vue` dans le fichier `router/index.ts`.
3. Ajouter un bouton dans la page `HomePage.vue`. Ce bouton permet de rediriger l'application vers la page `PageQuatre.vue`.

Exemple presque complet

- Créer un répertoire contenant l'application *Calculatrice*.

```
cd Workspace
```

```
ionic start
```

- Démarrer *ionic*

```
cd Calculatrice
```

```
ionic serve
```

- Un navigateur Web s'ouvre et la nouvelle application démarre
- Modifier le fichier `views/HomePage.vue` afin de créer une calculatrice.

```

1 <template>
2   <ion-page>
3     <ion-content color="dark" :fullscreen="true">

```

```

4         <ion-input v-model="equation" class="ion-text-end"></ion-input>
5         <ion-grid class="ion-text-center">
6             <ion-row>
7                 <ion-col>
8                     <ion-button shape="round" color="light">C</ion-button>
9                 </ion-col>
10                <ion-col>
11                    <ion-button shape="round" color="light">+/-</ion-button>
12                </ion-col>
13                <ion-col>
14                    <ion-button shape="round" color="light">&larr;</ion-button>
15                </ion-col>
16                <ion-col>
17                    <ion-button shape="round" color="warning">&#247;</ion-button>
18                </ion-col>
19            </ion-row>
20            <ion-row>
21                <ion-col>
22                    <ion-button shape="round" color="light"
23                        @click="addNum(7)">7</ion-button>
24                </ion-col>
25                <ion-col>
26                    <ion-button shape="round" color="light">8</ion-button>
27                </ion-col>
28                <ion-col>
29                    <ion-button shape="round" color="light">9</ion-button>
30                </ion-col>
31                <ion-col>
32                    <ion-button shape="round" color="warning">&#215;</ion-button>
33                </ion-col>
34            </ion-row>
35        </ion-grid>
36    </ion-content>
37 </ion-page>
38 </template>
39
40 <script lang="ts">
41 import {
42     IonContent,
43     IonPage,
44     IonInput,
45     IonGrid,
46     IonRow,
47     IonCol,
48     IonButton
49 } from '@ionic/vue';
50 import { defineComponent } from 'vue';
51
52 export default defineComponent({
53     name: 'HomePage',
54     components: {
55         IonContent,
56         IonPage,
57         IonInput,
58         IonGrid,

```

```

59         IonRow,
60         IonCol,
61         IonButton
62     },
63     methods: {
64         addNum(num: string) {
65             console.log(num);
66             this.equation += num;
67         }
68     },
69     data() {
70         return {
71             equation: ""
72         }
73     }
74 });
75 </script>
76
77 <style scoped>
78 ion-input {
79     font-size: 48px;
80     --padding-end: 20px;
81 }
82
83 ion-button {
84     height: 50px;
85     width: 50px;
86     border-radius: 50;
87     font-size: 20px;
88 }
89 </style>

```

Le système de grille d'Ionic est basé sur la flexbox, une fonctionnalité CSS prise en charge par tous les périphériques supportés par Ionic. La grille est composée de lignes et de colonnes. Les colonnes seront développées pour remplir leur ligne et seront redimensionnées pour s'adapter à des colonnes supplémentaires.

Il est maintenant temps de compléter cet exemple.

Cycle de vie

Ionic fournit quatre méthodes qui sont appelés à différent moment du cycle de vie d'un composant :

Nom de la méthode	Description
<code>ionViewWillEnter()</code>	S'exécute lorsque le composant vers lequel il est acheminé est sur le point d'être animé en vue. Puisque <code>ionViewWillEnter()</code> est appelé chaque fois que l'on navigue vers la vue (qu'elle soit initialisée ou non), c'est une bonne méthode pour charger des données à partir de services.
<code>ionViewDidEnter()</code>	S'exécute lorsque le composant vers lequel il est acheminé a fini de s'animer.

	Si vous constatez des problèmes de performance en utilisant <code>ionViewWillEnter()</code> lors du chargement des données, vous pouvez faire vos appels de données dans <code>ionViewDidEnter()</code> à la place. Toutefois, cet événement ne se déclenchera pas avant que la page ne soit visible pour l'utilisateur, de sorte que vous pourriez vouloir utiliser soit un indicateur de chargement, soit un écran squelette tel que <code><ion-skeleton-text></code> ou <code><ion-loading></code> , afin que le contenu ne clignote pas de manière non naturelle après la transition.
<code>ionViewWillLeave()</code>	S'exécute lorsque le composant d'où provient le routage est sur le point de s'animer. Peut-être utilisé pour le nettoyage, comme le désabonnement des sources de données.
<code>ionViewDidLeave()</code>	S'exécute lorsque le composant vers lequel il est acheminé a fini de s'animer.

Les méthodes associées au cycle de vie d'un composant doivent être définies dans la méthode `defineComponent()`.

```

1  ...
2
3  export default defineComponent({
4    name: 'Home',
5    ionViewDidEnter() {
6      console.log('Home page did enter');
7    },
8    ionViewDidLeave() {
9      console.log('Home page did leave');
10   },
11   ionViewWillEnter() {
12     console.log('Home page will enter');
13   },
14   ionViewWillLeave() {
15     console.log('Home page will leave');
16   },
17   components: {
18     IonPage
19   }
20 })
21
22  ...

```

Requête HTTP

Le protocole HTTP (Hypertext Transfer Protocol) est un protocole de communication client-serveur développé pour le Web. Ce protocole de la couche application utilise, dans les faits, le protocole TCP comme couche de transport.

Les clients HTTP les plus connus sont les navigateurs Web permettant à un utilisateur d'accéder à un serveur contenant les données. Un serveur HTTP utilise par défaut le port 80 (443 pour HTTPS).

Dans le protocole HTTP, il existe un mécanisme pour échanger des données entre le navigateur et le serveur Web. Ce mécanisme s'appelle requête HTTP. Il existe deux types de requêtes : synchrones et asynchrones.

Ionic, permet au programmeur d'effectuer des requêtes HTTP asynchrone à l'aide de l'instruction `fetch()`.

Exemple 5 : Récupérer des données en format JSON et les afficher

1. Créer un fichier `PageCinq.vue` dans le répertoire `views`.

```
1 <template>
2   <ion-page>
3     <ion-header>
4       <ion-toolbar color="primary">
5         <ion-buttons slot="start">
6           <ion-back-button default-href="/home"></ion-back-button>
7         </ion-buttons>
8         <ion-title>Page 5</ion-title>
9       </ion-toolbar>
10    </ion-header>
11
12    <ion-content>
13      <ion-item>
14        <ion-label>Titre album: </ion-label>
15        <ion-label>{{this.titre}}</ion-label>
16      </ion-item>
17    </ion-content>
18
19    <ion-footer>
20      <ion-toolbar color="secondary">
21        <ion-title>Pied de page</ion-title>
22      </ion-toolbar>
23    </ion-footer>
24  </ion-page>
25 </template>
26
27 <script lang="ts">
28 import {
29   IonContent,
30   IonHeader,
31   IonPage,
32   IonTitle,
33   IonToolbar,
34   IonFooter,
35   IonBackButton,
36   IonButtons,
37   IonLabel,
```

```

38     IonItem,
39 } from '@ionic/vue';
40 import { loadingController } from '@ionic/vue';
41 import { defineComponent } from 'vue';
42
43 export default defineComponent({
44   components: {
45     IonContent,
46     IonHeader,
47     IonPage,
48     IonTitle,
49     IonToolbar,
50     IonFooter,
51     IonBackButton,
52     IonButtons,
53     IonLabel,
54     IonItem,
55   },
56   data() {
57     return {
58       titre: "",
59     }
60   },
61   ionViewDidEnter() {
62     console.log('Home page did Enter');
63     this.getJSON();
64   },
65   methods: {
66     async getJSON() {
67       const loading = await loadingController
68         .create({
69           message: 'Attendre SVP...',
70         });
71       await loading.present();
72
73       let url = 'https://StephaneDGill.github.io/disque.json';
74       fetch(url)
75         .then(reponse => reponse.json())
76         .then(data => {
77           console.log(data);
78           this.titre = data.titre_album;
79           loading.dismiss();
80         });
81     }
82   }
83 });
84 </script>

```

2. Ajouter une route vers le fichier PageCinq.vue dans le fichier router/index.ts.
3. Ajouter un bouton dans la page HomePage.vue. Ce bouton permet de rediriger l'application vers la page PageCinq.vue.

Géolocalisation

Exemple 6 : Récupérer la position (longitude, latitude) d'un appareil

1. Installer le module de géolocalisation

```
npm install @capacitor/geolocation
```

```
ionic cap sync
```

2. Créer un fichier `PageSix.vue` dans le répertoire `views`.

```
1 <template>
2   <ion-page>
3     <ion-header>
4       <ion-toolbar color="primary">
5         <ion-buttons slot="start">
6           <ion-back-button default-href="/home"></ion-back-button>
7         </ion-buttons>
8         <ion-title>Page 6</ion-title>
9       </ion-toolbar>
10    </ion-header>
11
12    <ion-content>
13      <ion-item>
14        <ion-label>Longitude: </ion-label>
15        <ion-label>{{this.longitude}}</ion-label>
16      </ion-item>
17      <ion-item>
18        <ion-label>Latitude: </ion-label>
19        <ion-label>{{this.latitude}}</ion-label>
20      </ion-item>
21    </ion-content>
22
23    <ion-footer>
24      <ion-toolbar color="secondary">
25        <ion-title>Pied de page</ion-title>
26      </ion-toolbar>
27    </ion-footer>
28  </ion-page>
29 </template>
30
31 <script lang="ts">
32 import {
33   IonContent,
34   IonHeader,
35   IonPage,
36   IonTitle,
37   IonToolbar,
```

```

38     IonFooter,
39     IonBackButton,
40     IonButtons,
41     IonLabel,
42     IonItem,
43 } from '@ionic/vue';
44 import { loadingController } from '@ionic/vue';
45 import { defineComponent } from 'vue';
46 import { Geolocation } from '@capacitor/geolocation';
47
48 export default defineComponent({
49   components: {
50     IonContent,
51     IonHeader,
52     IonPage,
53     IonTitle,
54     IonToolbar,
55     IonFooter,
56     IonBackButton,
57     IonButtons,
58     IonLabel,
59     IonItem,
60   },
61   data() {
62     return {
63       latitude: 0,
64       longitude: 0,
65     }
66   },
67   ionViewDidEnter() {
68     console.log('Home page did Enter');
69     this.getCurrentPosition();
70   },
71   methods: {
72     async getCurrentPosition() {
73       const coordinates = await Geolocation.getCurrentPosition();
74       console.log('Current', coordinates);
75       this.latitude = coordinates.coords.latitude;
76       this.longitude = coordinates.coords.longitude;
77     },
78   }
79 });
80 </script>
81

```

3. Ajouter une route vers le fichier PageSix.vue dans le fichier router/index.ts.
4. Ajouter un bouton dans la page HomePage.vue. Ce bouton permet de rediriger l'application vers la page PageSix.vue.

Images

Exemple 7 : Récupérer des données en format JSON et les afficher

1. Créer un fichier PageSept .vue dans le répertoire views.

```
1 <template>
2   <ion-page>
3     <ion-header>
4       <ion-toolbar color="primary">
5         <ion-buttons slot="start">
6           <ion-back-button default-href="/home"></ion-back-button>
7         </ion-buttons>
8         <ion-title>Page 7</ion-title>
9       </ion-toolbar>
10    </ion-header>
11
12    <ion-content>
13      <ion-item>
14        <ion-img src="assets/img/01d.png"></ion-img>
15      </ion-item>
16      <ion-item>
17        <ion-img :src="this.imageSrc"></ion-img>
18      </ion-item>
19    </ion-content>
20
21    <ion-footer>
22      <ion-toolbar color="secondary">
23        <ion-title>Pied de page</ion-title>
24      </ion-toolbar>
25    </ion-footer>
26  </ion-page>
27 </template>
28
29 <script lang="ts">
30 import {
31   IonContent,
32   IonHeader,
33   IonPage,
34   IonTitle,
35   IonToolbar,
36   IonFooter,
37   IonBackButton,
38   IonButtons,
39   IonItem,
40   IonImg,
41 } from '@ionic/vue';
42 import { loadingController } from '@ionic/vue';
43 import { defineComponent } from 'vue';
44
45 export default defineComponent({
46   components: {
47     IonContent,
```

```

48         IonHeader,
49         IonPage,
50         IonTitle,
51         IonToolbar,
52         IonFooter,
53         IonBackButton,
54         IonButtons,
55         IonItem,
56         IonImg,
57     },
58     data() {
59         return {
60             imageSrc: "http://openweathermap.org/img/wn/10d@2x.png",
61         }
62     },
63 });
64 </script>

```

2. Ajouter une route vers le fichier `PageSept.vue` dans le fichier `router/index.ts`.
3. Ajouter un bouton dans la page `HomePage.vue`. Ce bouton permet de rediriger l'application vers la page `PageSept.vue`.

Menu de navigation coulissant

Le menu de navigation coulissant est une tendance courante dans les applications Web ou mobile. Sous Ionic, ce menu apparaît à gauche.

Exemple 8 : Menu de navigation coulissant.

1. Créer un nouveau projet puis modifier le fichier `App.vue` dans le répertoire `src` afin d'ajouter un menu de navigation coulissant.

```

1 <template>
2   <ion-app>
3     <ion-split-pane content-id="main-content">
4       <ion-menu content-id="main-content" type="overlay">
5         <ion-content>
6           <ion-list>
7             <ion-list-header>Mon menu</ion-list-header>
8             <ion-menu-toggle>
9               <ion-item>
10                <ion-icon slot="start" :ios="logoTux"
11                  :md="logoTux"></ion-icon>
12                <ion-label>Page 1</ion-label>
13              </ion-item>
14              <ion-item>
15                <ion-icon slot="start" :ios="logoTux"
16                  :md="logoTux"></ion-icon>

```

```

17         <ion-label>Page 2</ion-label>
18     </ion-item>
19 </ion-menu-toggle>
20 </ion-list>
21 </ion-content>
22 </ion-menu>
23 <ion-router-outlet id="main-content"></ion-router-outlet>
24 </ion-split-pane>
25 </ion-app>
26 </template>
27
28 <script lang="ts">
29 import {
30     IonApp,
31     IonRouterOutlet,
32     IonSplitPane,
33     IonContent,
34     IonList,
35     IonListHeader,
36     IonItem,
37     IonLabel,
38     IonMenu,
39     IonMenuToggle,
40     IonIcon,
41 } from '@ionic/vue';
42 import { defineComponent } from 'vue';
43 // Importer les icones
44 import { logoTux } from 'ionicons/icons';
45
46 export default defineComponent({
47     name: 'App',
48     components: {
49         IonApp,
50         IonRouterOutlet,
51         IonSplitPane,
52         IonContent,
53         IonList,
54         IonListHeader,
55         IonItem,
56         IonLabel,
57         IonMenu,
58         IonMenuToggle,
59         IonIcon,
60     },
61     setup() {
62         return {
63             logoTux
64         }
65     }
66 });
67 </script>

```

2. Modifier le fichier `views/HomePage.vue` afin d'ajouter un icône qui permet d'ouvrir le menu.

```

1 <template>
2   <ion-page>
3     <ion-header :translucent="true">
4       <ion-toolbar>
5         <ion-buttons slot="start">
6           <ion-menu-button color="primary"></ion-menu-button>
7         </ion-buttons>
8         <ion-title>Page d'accueil</ion-title>
9       </ion-toolbar>
10    </ion-header>
11
12    <ion-content :fullscreen="true">
13      <ion-header collapse="condense">
14        <ion-toolbar>
15          <ion-title size="large">Page d'accueil</ion-title>
16        </ion-toolbar>
17      </ion-header>
18
19      <div id="container">
20        <strong>Ready to create an app?</strong>
21        <p>Start with Ionic <a target="_blank" rel="noopener...
22      </div>
23    </ion-content>
24  </ion-page>
25 </template>
26
27 <script lang="ts">
28 import {
29   IonContent,
30   IonHeader,
31   IonPage,
32   IonTitle,
33   IonToolbar,
34   IonButtons,
35   IonMenuButton,
36 } from '@ionic/vue';
37 import { defineComponent } from 'vue';
38
39 export default defineComponent({
40   name: 'HomePage',
41   components: {
42     IonContent,
43     IonHeader,
44     IonPage,
45     IonTitle,
46     IonToolbar,
47     IonButtons,
48     IonMenuButton,
49   }
50 });
51 </script>
52 ...

```

Exemple 9 : Ajout de la navigation

1. Créer un fichier PageDeux .vue dans le répertoire views.

```
1  <template>
2    <ion-page>
3      <ion-header>
4        <ion-toolbar color="primary">
5          <ion-title>Page 2</ion-title>
6        </ion-toolbar>
7      </ion-header>
8
9      <ion-content>
10       <div class="ion-text-center milieu">
11         <p>Page 2</p>
12       </div>
13     </ion-content>
14
15     <ion-footer>
16       <ion-toolbar color="secondary">
17         <ion-title>Pied de page</ion-title>
18       </ion-toolbar>
19     </ion-footer>
20   </ion-page>
21 </template>
22
23 <script lang="ts">
24 import {
25   IonContent,
26   IonHeader,
27   IonPage,
28   IonTitle,
29   IonToolbar,
30   IonFooter
31 } from '@ionic/vue';
32 import { defineComponent } from 'vue';
33
34 export default defineComponent({
35   components: {
36     IonContent,
37     IonHeader,
38     IonPage,
39     IonTitle,
40     IonToolbar,
41     IonFooter
42   },
43 });
44 </script>
45
46 <style scoped>
47 .milieu {
48   height: 100%;
49   display: flex;
50   justify-content: center;
```

```

51     align-items: center;
52 }
53 </style>

```

2. Ajouter une route vers le fichier `PageDeux.vue` dans le fichier `router/index.ts`.

```

1  ...
2  {
3      path: '/PageDeux/:id',
4      name: 'PageDeux',
5      component: PageDeux
6  }
7  ...

```

À la ligne 3, le symbole `:id` permet d'ajouter un paramètre à la route

3. Ajouter un lien vers la page `PageDeux.vue` en modifiant la ligne 14 du fichier `src/App.vue`.

```

1  <ion-item router-link="/PageDeux/1">

```

C'est la valeur 1 qui sera passé en paramètre.

4. Dans l'entête de la `PageDeux.vue` ajouter un bouton de retour à la page précédente et affiché le paramètre

```

2  <template>
3      <ion-page>
4          <ion-header>
5              <ion-toolbar color="primary">
6                  <ion-buttons slot="start">
7                      <ion-back-button @click="this.router.back()">
8                      </ion-back-button>
9                  </ion-buttons>
10                 <ion-title>Page 2</ion-title>
11             </ion-toolbar>
12         </ion-header>
13
14         <ion-content>
15             <div class="ion-text-center milieu">
16                 <p>Page 2; Param: {{this.route.params.id}}</p>
17             </div>
18         </ion-content>
19
20         <ion-footer>
21             <ion-toolbar color="secondary">
22                 <ion-title>Pied de page</ion-title>
23             </ion-toolbar>
24         </ion-footer>
25     </ion-page>
26 </template>
27
28 <script lang="ts">
29 import {
30     IonContent,

```

```

31     IonHeader,
32     IonPage,
33     IonTitle,
34     IonToolbar,
35     IonFooter,
36     IonButtons,
37     IonBackButton,
38 } from '@ionic/vue';
39 import { defineComponent } from 'vue';
40 import { useRoute, useRouter } from 'vue-router';
41
42 export default defineComponent({
43   components: {
44     IonContent,
45     IonHeader,
46     IonPage,
47     IonTitle,
48     IonToolbar,
49     IonFooter,
50     IonButtons,
51     IonBackButton,
52   },
53   setup() {
54     const route = useRoute();
55     const router = useRouter();
56     return { route, router };
57   },
58 });
59 </script>
60
61 <style scoped>
62 .milieu {
63   height: 100%;
64   display: flex;
65   justify-content: center;
66   align-items: center;
67 }
68 </style>

```

Affichage de liste

Exemple 10 : Afficher le contenu d'un tableau d'objet

1. Créer un fichier `PageHuit.vue` dans le répertoire `views`.

```

1 <template>
2   <ion-page>
3     <ion-header>
4       <ion-toolbar color="primary">

```

```

5         <ion-buttons slot="start">
6             <ion-back-button default-href="/home"></ion-back-button>
7         </ion-buttons>
8         <ion-title>Page 8</ion-title>
9     </ion-toolbar>
10 </ion-header>
11
12     <ion-content>
13         <div class="ion-text-center">
14             <h1>Ma liste</h1>
15             <p v-for="item in this.maListe" :key="item.id">{{item.nom}}</p>
16         </div>
17     </ion-content>
18
19     <ion-footer>
20         <ion-toolbar color="secondary">
21             <ion-title>Pied de page</ion-title>
22         </ion-toolbar>
23     </ion-footer>
24 </ion-page>
25 </template>
26
27 <script lang="ts">
28 import {
29     IonContent,
30     IonHeader,
31     IonPage,
32     IonTitle,
33     IonToolbar,
34     IonFooter,
35     IonBackButton,
36     IonButtons
37 } from '@ionic/vue';
38 import { defineComponent } from 'vue';
39
40 export default defineComponent({
41     components: {
42         IonContent,
43         IonHeader,
44         IonPage,
45         IonTitle,
46         IonToolbar,
47         IonFooter,
48         IonBackButton,
49         IonButtons,
50     },
51     data() {
52         return {
53             maListe: [{id: 1, nom: "Stephane"},
54                       {id: 2, nom: "Nicolas"},
55                       {id: 3, nom: "Nathalie"}],
56         }
57     },
58 });
59 </script>

```

À la ligne 15, l'attribut v-for de la balise <p> permet de parcourir le tableau d'objet.

2. Ajouter une route vers le fichier PageHuit .vue dans le fichier router/index .ts.
3. Ajouter un bouton dans la page HomePage .vue. Ce bouton permet de rediriger l'application vers la page PageHuit .vue.

Exemple 11 : Afficher le contenu d'un tableau d'objet reçu en format JSON

1. Créer un fichier PageNeuf .vue dans le répertoire views.

```
1 <template>
2   <ion-page>
3     <ion-header>
4       <ion-toolbar color="primary">
5         <ion-buttons slot="start">
6           <ion-back-button default-href="/home"></ion-back-button>
7         </ion-buttons>
8         <ion-title>Page 9</ion-title>
9       </ion-toolbar>
10    </ion-header>
11
12    <ion-content>
13      <div class="ion-text-center">
14        <h1>Ma liste</h1>
15        <p v-for="item in this.maListe" :key="item.id">{{item.nom}}</p>
16      </div>
17    </ion-content>
18
19    <ion-footer>
20      <ion-toolbar color="secondary">
21        <ion-title>Pied de page</ion-title>
22      </ion-toolbar>
23    </ion-footer>
24  </ion-page>
25 </template>
26
27 <script lang="ts">
28 import {
29   IonContent,
30   IonHeader,
31   IonPage,
32   IonTitle,
33   IonToolbar,
34   IonFooter,
35   IonBackButton,
36   IonButtons
37 } from '@ionic/vue';
38 import { defineComponent } from 'vue';
39
40 export default defineComponent({
41   components: {
```

```

42         IonContent,
43         IonHeader,
44         IonPage,
45         IonTitle,
46         IonToolbar,
47         IonFooter,
48         IonBackButton,
49         IonButtons,
50     },
51     data() {
52         return {
53             maliste: [{}],
54         }
55     },
56     ionViewDidEnter() {
57         this.getJSON();
58     },
59     methods: {
60         async getJSON() {
61             let url = 'https://StephaneDGill.github.io/nom.json';
62             fetch(url)
63                 .then(reponse => response.json())
64                 .then(data => {
65                     console.log(data);
66                     let temp: object[] = [];
67                     for (let i = 0; i < data.length; i++) {
68                         temp.push({ id: i, nom: data[i].nom });
69                     }
70                     this.maliste = temp;
71                 });
72         }
73     }
74 });
75 </script>

```

2. Ajouter une route vers le fichier `PageNeuf.vue` dans le fichier `router/index.ts`.
3. Ajouter un bouton dans la page `HomePage.vue`. Ce bouton permet de rediriger l'application vers la page `PageNeuf.vue`.

Stockage local

Exemple 12 : Enregistrer la date et l'heure à laquelle on presse le bouton.

1. Installer les modules nécessaires

```
npm install @capacitor/storage
```

ionic cap sync

2. Créer un fichier PageDix.vue dans le répertoire views.

```
1 <template>
2   <ion-page>
3     <ion-header>
4       <ion-toolbar color="primary">
5         <ion-buttons slot="start">
6           <ion-back-button default-href="/home"></ion-back-button>
7         </ion-buttons>
8         <ion-title>Page 10</ion-title>
9       </ion-toolbar>
10    </ion-header>
11
12    <ion-content>
13      <div class="ion-text-center">
14        <p>{{this.heure}}</p>
15        <ion-button color="medium" @click="this.setHeure()">
16          Set heure</ion-button>
17      </div>
18    </ion-content>
19
20    <ion-footer>
21      <ion-toolbar color="secondary">
22        <ion-title>Pied de page</ion-title>
23      </ion-toolbar>
24    </ion-footer>
25  </ion-page>
26 </template>
27
28 <script lang="ts">
29 import {
30   IonContent,
31   IonHeader,
32   IonPage,
33   IonTitle,
34   IonToolbar,
35   IonFooter,
36   IonBackButton,
37   IonButtons,
38   IonButton
39 } from '@ionic/vue';
40 import { defineComponent } from 'vue';
41 import { Storage } from '@capacitor/storage';
42
43 export default defineComponent({
44   components: {
45     IonContent,
46     IonHeader,
47     IonPage,
48     IonTitle,
49     IonToolbar,
50     IonFooter,
```

```

51         IonBackButton,
52         IonButtons,
53         IonButton
54     },
55     data() {
56         return {
57             heure: " ",
58         }
59     },
60     ionViewDidEnter() {
61         this.getHeure();
62     },
63     methods: {
64         async setHeure() {
65             const heure = String(new Date().toLocaleString());
66             await Storage.set({
67                 key: 'heure',
68                 value: heure,
69             });
70             this.getHeure();
71         },
72         async getHeure() {
73             const { value } = await Storage.get({ key: 'heure' });
74             console.log(`Heure: ${value}`);
75             this.heure = String(value);
76         }
77     }
78 });
79
80 </script>
81
82 <style scoped>
83 .milieu {
84     height: 100%;
85     display: flex;
86     justify-content: center;
87     align-items: center;
88 }
89 </style>

```

3. Ajouter une route vers le fichier `PageDix.vue` dans le fichier `router/index.ts`.
4. Ajouter un bouton dans la page `PageDix.vue`. Ce bouton permet de rediriger l'application vers la page `PageSept.vue`.

Module

Exemple 13 : Enregistrer la date et l'heure à laquelle on presse le bouton.

1. Créer un module `stockageHeure.ts` dans le dossier `src/composables`.

```

1 import { Storage } from '@capacitor/storage';
2
3 export function stockageHeure() {
4     const setHeure = async (): Promise<string> => {
5         const heure = String(new Date().toLocaleString());
6         await Storage.set({
7             key: 'heure',
8             value: heure,
9         });
10        return (await getHeure());
11    };
12
13    const getHeure = async (): Promise<string> => {
14        const { value } = await Storage.get({ key: 'heure' });
15        console.log(`Heure: ${value}`);
16        return(String(value));
17    };
18
19    return {
20        setHeure,
21        getHeure,
22    };
23 }

```

2. Créer un fichier PageOnze .vue dans le répertoire views.

```

1 <template>
2   <ion-page>
3     <ion-header>
4       <ion-toolbar color="primary">
5         <ion-buttons slot="start">
6           <ion-back-button default-href="/home"></ion-back-button>
7         </ion-buttons>
8         <ion-title>Page 11</ion-title>
9       </ion-toolbar>
10    </ion-header>
11
12    <ion-content>
13      <div class="ion-text-center">
14        <p>{{this.heureAff}}</p>
15        <ion-button color="medium" @click="this.action()">
16          Set heure</ion-button>
17      </div>
18    </ion-content>
19
20    <ion-footer>
21      <ion-toolbar color="secondary">
22        <ion-title>Pied de page</ion-title>
23      </ion-toolbar>
24    </ion-footer>
25  </ion-page>
26 </template>
27
28 <script lang="ts">
29 import {

```

```

30     IonContent,
31     IonHeader,
32     IonPage,
33     IonTitle,
34     IonToolbar,
35     IonFooter,
36     IonBackButton,
37     IonButtons,
38     IonButton
39 } from '@ionic/vue';
40 import { defineComponent } from 'vue';
41 import { stockageHeure } from '@/composables/stockageHeure';
42
43 export default defineComponent({
44   components: {
45     IonContent,
46     IonHeader,
47     IonPage,
48     IonTitle,
49     IonToolbar,
50     IonFooter,
51     IonBackButton,
52     IonButtons,
53     IonButton
54   },
55   data() {
56     return {
57       heureAff: " ",
58     }
59   },
60   setup() {
61     const { setHeure, getHeure } = stockageHeure();
62
63     return {
64       setHeure,
65       getHeure
66     }
67   },
68   async ionViewDidEnter() {
69     this.heureAff = await this.getHeure();
70   },
71   methods: {
72     async action() {
73       this.heureAff = await this.setHeure();
74     }
75   }
76 });
77
78
79 </script>

```

3. Ajouter une route vers le fichier PageOnze.vue dans le fichier router/index.ts.

4. Ajouter un bouton dans la page `HomePage.vue`. Ce bouton permet de rediriger l'application vers la page `PageOnze.vue`.

Camera

1. Installer les modules nécessaires

```
npm install @capacitor/camera
```

```
npm install @ionic/pwa-elements
```

```
ionic cap sync
```

2. Modifier le fichier `main.ts` en ajoutant les instructions suivantes :

```
1 import { defineCustomElements } from '@ionic/pwa-elements/loader';
2 defineCustomElements(window);
```

3. Créer un fichier `PageDouze.vue` dans le répertoire `views` du projet Composant.

```
1 <template>
2   <ion-page>
3     <ion-header>
4       <ion-toolbar color="primary">
5         <ion-buttons slot="start">
6           <ion-back-button default-href="/home"></ion-back-button>
7         </ion-buttons>
8         <ion-title>Page 12</ion-title>
9       </ion-toolbar>
10    </ion-header>
11
12    <ion-content>
13      <div class="ion-text-center">
14        <p>Camera</p>
15        <ion-img :src="this.imageSrc"></ion-img>
16        <ion-button @click="prendrePhoto">Prendre Photo</ion-button>
17      </div>
18    </ion-content>
19
20    <ion-footer>
21      <ion-toolbar color="secondary">
22        <ion-title>Pied de page</ion-title>
23      </ion-toolbar>
24    </ion-footer>
25  </ion-page>
26 </template>
27
28 <script lang="ts">
29 import {
30   IonContent,
31   IonHeader,
```

```

32     IonPage,
33     IonTitle,
34     IonToolbar,
35     IonFooter,
36     IonBackButton,
37     IonButtons,
38     IonButton,
39     IonImg,
40 } from '@ionic/vue';
41 import { defineComponent } from 'vue';
42
43 import { Camera, CameraResultType } from '@capacitor/camera';
44
45 export default defineComponent({
46   components: {
47     IonContent,
48     IonHeader,
49     IonPage,
50     IonTitle,
51     IonToolbar,
52     IonFooter,
53     IonBackButton,
54     IonButtons,
55     IonButton,
56     IonImg,
57   },
58   methods: {
59     async prendrePhoto() {
60       console.log("Prendre photo");
61       const image = await Camera.getPhoto({
62         quality: 90,
63         allowEditing: true,
64         resultType: CameraResultType.Uri
65       });
66       this.imageSrc = String(image.webPath);
67     }
68   },
69   data() {
70     return {
71       imageSrc: "assets/img/NoImage.png",
72     }
73   },
74 });
75 </script>

```

4. Ajouter une route vers le fichier PageDouze .vue dans le fichier router/index .ts.
5. Ajouter un bouton dans la page HomePage .vue. Ce bouton permet de rediriger l'application vers la page PageDouze .vue.

Menu de navigation à onglet

Exemple 14 : Menu de navigation à onglet

```
1  <template>
2    <ion-page>
3      <ion-tabs>
4        <ion-router-outlet></ion-router-outlet>
5        <ion-tab-bar slot="bottom">
6          <ion-tab-button tab="tab1" href="/tabs/tab1">
7            <ion-icon :icon="triangle" />
8            <ion-label>Tab 1</ion-label>
9          </ion-tab-button>
10
11          <ion-tab-button tab="tab2" href="/tabs/tab2">
12            <ion-icon :icon="ellipse" />
13            <ion-label>Tab 2</ion-label>
14          </ion-tab-button>
15
16          <ion-tab-button tab="tab3" href="/tabs/tab3">
17            <ion-icon :icon="square" />
18            <ion-label>Tab 3</ion-label>
19          </ion-tab-button>
20        </ion-tab-bar>
21      </ion-tabs>
22    </ion-page>
23  </template>
24
25  <script lang="ts">
26    import { defineComponent } from 'vue';
27    import {
28      IonTabBar,
29      IonTabButton,
30      IonTabs,
31      IonLabel,
32      IonIcon,
33      IonPage,
34      IonRouterOutlet
35    } from '@ionic/vue';
36    import { ellipse, square, triangle } from 'ionicons/icons';
37
38    export default defineComponent({
39      name: 'TabsPage',
40      components: {
41        IonLabel,
42        IonTabs,
43        IonTabBar,
44        IonTabButton,
45        IonIcon,
```

```
46         IonPage,  
47         IonRouterOutlet  
48     },  
49     setup() {  
50         return {  
51             ellipse,  
52             square,  
53             triangle,  
54         }  
55     }  
56 });  
57 </script>
```

Publier sur Google Play

Référence : <https://ionicframework.com/docs/deployment/play-store>

Exemple 15 : Créer une version installable sur un téléphone

```
1  # Étape 1: À partir du répertoire racine du projet taper  
2  # la commande suivante:  
3  ionic cap build android --no-open  
4  
5  # Étape 2: Aller dans le répertoire Android  
6  cd android  
7  
8  # Étape 3: Créer le "debug build"  
9  ./gradlew assembleDebug  
10  
11 # Étape 4: Créer le "release build"  
12 ./gradlew assembleRelease
```

Le fichier "apk" qui peut être installé sur votre téléphone Android se trouve dans le répertoire `android/app/build/outputs/apk/release/`

Références

- <https://ionicframework.com/docs/>

- <https://capacitorjs.com/docs>